

PROGRAMMATION FONCTIONNELLE 2 : TD3

janvier 2026 — Pierre Rousselin

Exercice 1 : Incrémenter, décrémenter

On considère la fonction OCaml

```
let foo = fun x -> fun y -> if x then y + 1 else y - 1
```

1. Quel est le type de `foo` ?
2. Que vaut `foo false` ? Quel est son type ?
3. Que vaut `foo true 41` ? Quel est son type ?
4. Que vaut `fun b -> foo b 42` ? Quel est son type ?
5. Que vaut `fun b -> foo b 0 = 1` ? Quel est son type ?
6. Écrire une définition équivalente de `foo` utilisant :
 - a) un seul `fun`
 - b) aucun `fun` (contrairement à cet exercice, huhuhu).

--- * ---

Exercice 2 : Curryfier, décurryfier

On considère la fonction OCaml

```
let curry = fun f -> fun x -> fun y -> f (x, y)
```

1. Donner le type de la fonction `curry`. On pourra utiliser les variables de type `'a`, `'b`, ...
2. Définir la fonction réciproque `uncurry` et donner son type.
3. Donner plusieurs définitions équivalentes à `uncurry foo`, sans utiliser `uncurry`, où `foo` est la fonction de l'exercice précédent.

--- * ---

Exercice 3 : Terme de preuve

Donner 2 preuves de chacun des lemmes suivants : une preuve avec juste `exact` et une preuve utilisant `apply`, `destruct`, `split`, etc.

- `Lemma td3ex3a (A B C : Prop) : A -> B -> (A -> B -> C) -> C.`
- `Lemma td3ex3b (A B C : Prop) : (A -> B -> C) -> (A /\ B -> C).`
- `Lemma td3ex3c (A B C : Prop) : (A /\ B -> C) -> (A -> B -> C).`

Que dire des preuves de `td3ex3b` et `td3ex3c` ?

--- * ---

Exercice 4 : Dédution naturelle

En fonction des besoins et du temps restant dans le TD, vous pouvez prouver les lemmes de l'exercice précédent avec des arbres de preuves en déduction naturelle.

--- * ---